



University
of Glasgow

W8-02: HBase

COMPSCI4064 (H) and COMPSCI5088 (M)

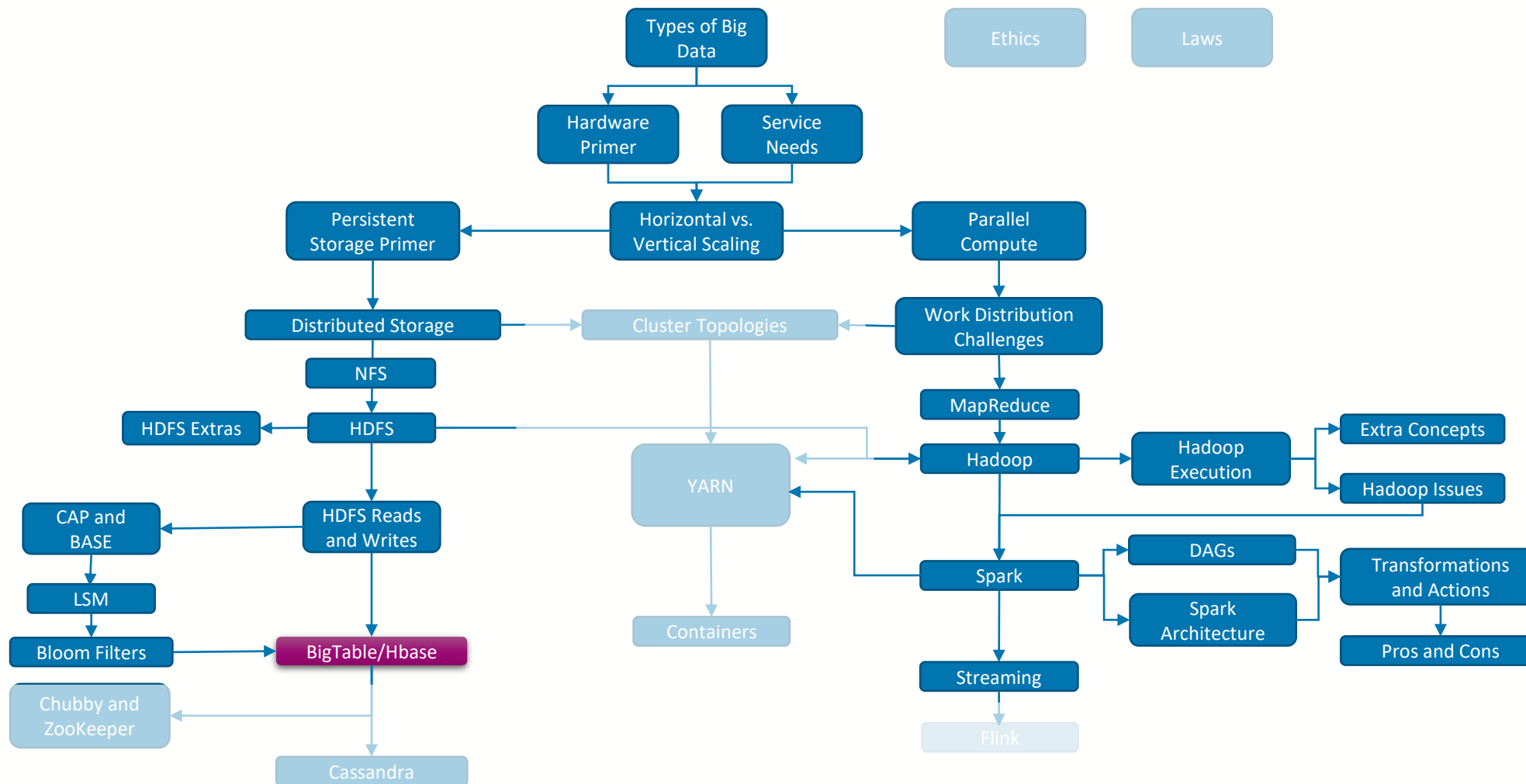
Dr. Richard McCreddie

**WORLD
CHANGING
GLASGOW**

**A WORLD
TOP 100
UNIVERSITY**



Where are we?





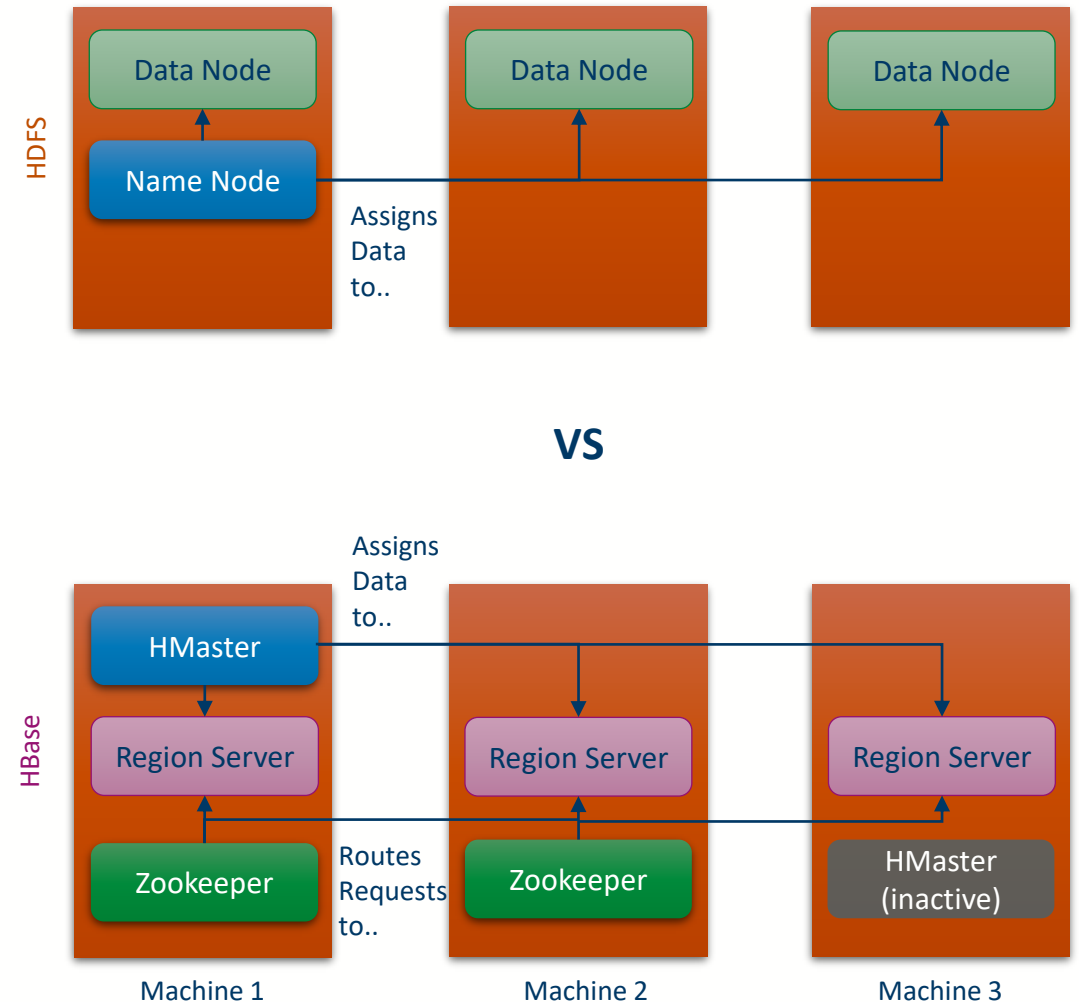
HBase

- HBase is an **open-source non-relational distributed database**
 - Explicitly modeled after Google's BigTable, written in Java
 - Focuses on **Consistency** and **Availability**
- Provides a table-like abstraction but it is not a RDBMS
 - No transactions
 - No normalization
 - No relational API
 - No proper schema
- Aims to maintain the **fast read speeds of the HDFS**, while providing **better write and append throughput** for records



HBase Architecture

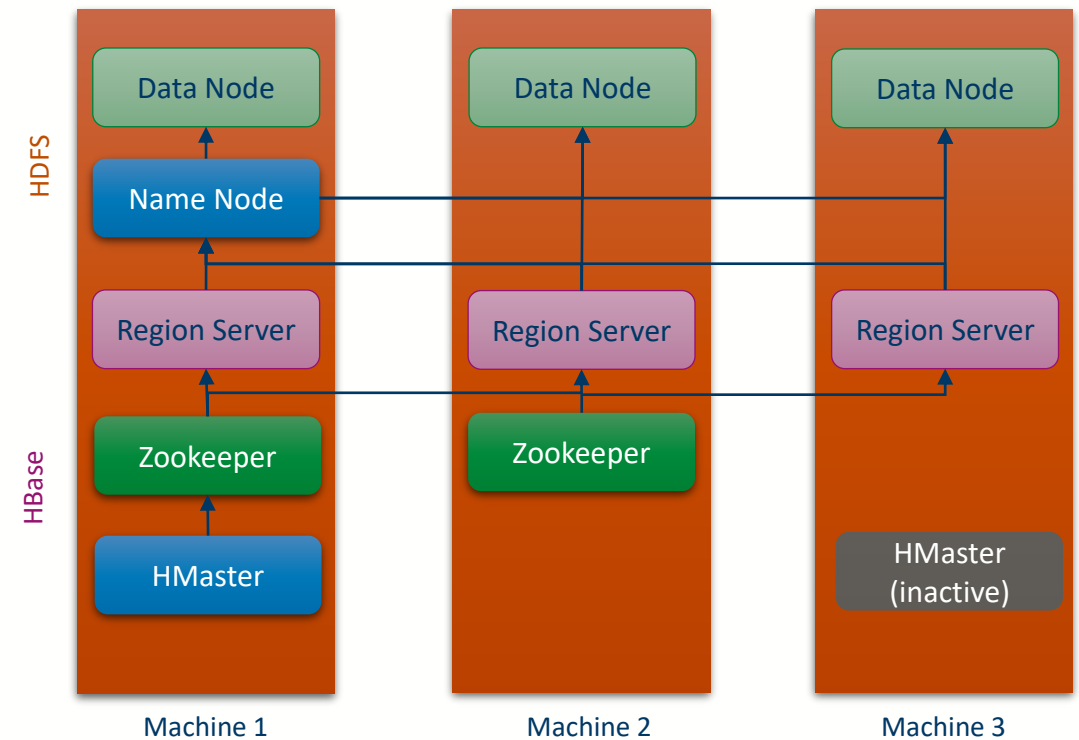
- HBase
 - **HMaster**: Assigns regions to Region Server. Creates/deletes tables. Routes data requests to region servers. Handle failures.
 - **Region Server**: Handles data requests from the user.
 - **Zookeeper**: Acts as a communication bridge. Tracks state of HMaster and Region Servers. Triggers recovery.





HBase uses the HDFS?

- When HBase finally needs to write a file to disk it write to the HDFS
 - ... i.e. the region server acts as a client communicating to the Name Node
 - ... and then writes its data to the listed Data Nodes





HBase Data Model (its like BigTable)

A **row** is a grouping of cells that share a **RowKey**

RowKey	Name	Degree	Timestamp
000001	David	MSc CS	21/11/2022...
000002	Jing	MSc CS	21/11/2022...
000003	Xi	MSc DS	21/11/2022...
000004	John	MSc DS	29/11/2022...
000005	Ellen		21/11/2022...
000006	Yang		21/11/2022...
000007	Misbah	MSci Joint	21/11/2022...
000008	Simona	MSci	26/11/2022...

HBase Data Model (its like BigTable)

Columns store the data for each RowKey. One RowKey can have unlimited columns

RowKey	Name	Degree	Timestamp
000001	David	MSc CS	21/11/2022...
000002	Jing	MSc CS	21/11/2022...
000003	Xi	MSc DS	21/11/2022...
000004	John	MSc DS	29/11/2022...
000005	Ellen		21/11/2022...
000006	Yang		21/11/2022...
000007	Misbah	MSci Joint	21/11/2022...
000008	Simona	MSci	26/11/2022...

HBase Data Model (its like BigTable)

Columns can be grouped into **column families** that can be read together (used for storage optimization)

RowKey	Name	Degree	Timestamp
000001	David	MSc CS	21/11/2022...
000002	Jing	MSc CS	21/11/2022...
000003	Xi	MSc DS	21/11/2022...
000004	John	MSc DS	29/11/2022...
000005	Ellen		21/11/2022...
000006	Yang		21/11/2022...
000007	Misbah	MSci Joint	21/11/2022...
000008	Simona	MSci	26/11/2022...

Column Family

HBase Data Model (its like BigTable)

Columns can be referenced by index or by a **column qualifier**, which are column titles

Column Qualifiers

RowKey	Name	Degree	Timestamp
000001	David	MSc CS	21/11/2022...
000002	Jing	MSc CS	21/11/2022...
000003	Xi	MSc DS	21/11/2022...
000004	John	MSc DS	29/11/2022...
000005	Ellen		21/11/2022...
000006	Yang		21/11/2022...
000007	Misbah	MSci Joint	21/11/2022...
000008	Simona	MSci	26/11/2022...

HBase Data Model (its like BigTable)

A cell in the table is a combination of the RowKey, Column Qualifier

RowKey	Name	Degree	Timestamp
000001	David	MSc CS	21/11/2022...
000002	Jing	MSc CS	21/11/2022...
000003	Xi	MSc DS	21/11/2022...
000004	John	MSc DS	29/11/2022...
000005	Ellen		21/11/2022...
000006	Yang		21/11/2022...
000007	Misbah	MSci Joint	21/11/2022...
000008	Simona	MSci	26/11/2022...

HBase Data Model (its like BigTable)

RowKey	Name	Degree	Timestamp
000001	David	MSc CS	21/11/2022...
000002	Jing	MSc CS	21/11/2022...
000003	Xi	MSc DS	21/11/2022...
000004	John	MSc DS	29/11/2022...
000005	Ellen		21/11/2022...
000006	Yang		21/11/2022...
000007	Misbah	MSci Joint	21/11/2022...
000008	Simona	MSci	26/11/2022...

When data is stored a **timestamp** is also recorded

When data is updated the new data is written with a new timestamp

Multiple versions of a row might exist with different timestamps

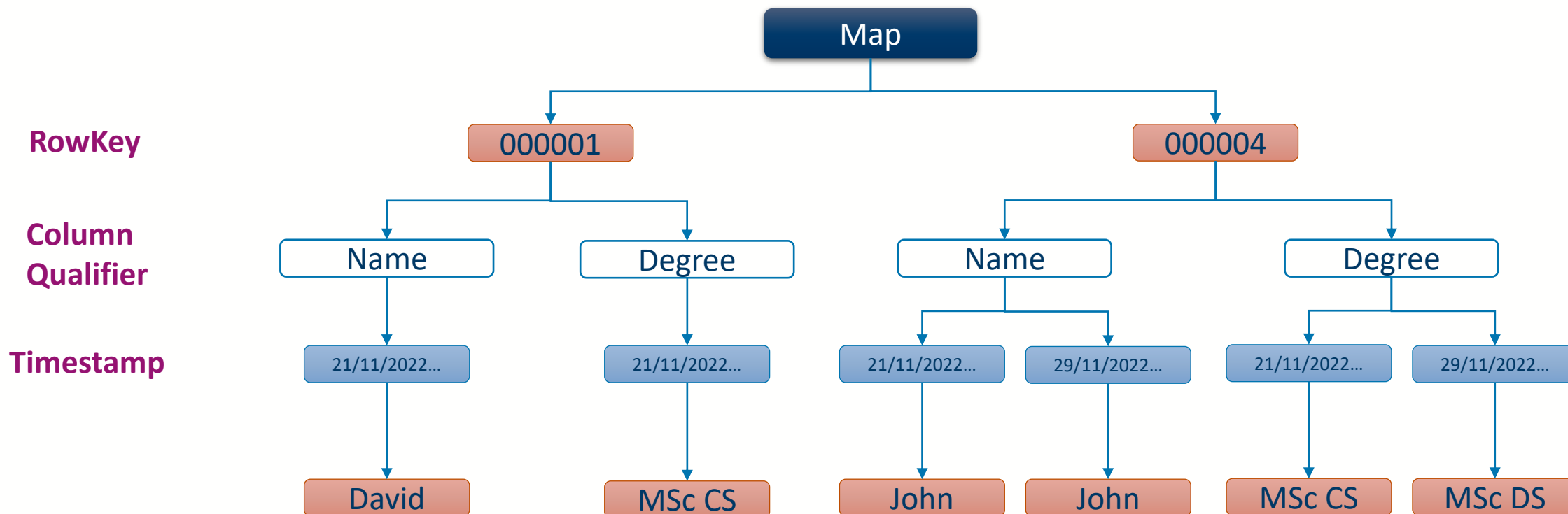
HBase Data Model (its like BigTable)

Regions are contiguous sets of rows that are stored together

RowKey	Name	Degree	Timestamp
000001	David	MSc CS	21/11/2022...
000002	Jing	MSc CS	21/11/2022...
000003	Xi	MSc DS	21/11/2022...
000004	John	MSc DS	29/11/2022...
000005	Ellen		21/11/2022...
000006	Yang		21/11/2022...
000007	Misbah	MSci Joint	21/11/2022...
000008	Simona	MSci	26/11/2022...

Data Addressing

- While it is conceptually useful to think about HBase as a table storage mechanism, internally data is referenced as a **multi-dimensional map**





Data Addressing

- While it is conceptually useful to think about HBase as a table storage mechanism, internally data is referenced as a **multi-dimensional map**
- In practice, its better to think of it as a **key-value store**:
 - **Key**: <RowKey, Column Qualifier, Timestamp>
 - **Value**: <value>
- So... there is no such thing as a “row” in the traditional sense
 - A row is simply the set of all cells of a given table sharing the same rowkey



Ordering and Grouping

- Rows are split into contiguous groups called **regions**
 - Regions are assigned by the HMaster, and managed by a Region Server
 - Held in memory within the Region Server until written to disk as an '**HFile**'
- Ordering:
 - Rows within a **region** are **ordered lexicographically** when written to disk
 - Sort happens just before write
 - Within each **row**, cells sorted **lexicographically on column qualifiers**
 - Within each set of **cells** with the same column qualifier, cells **sorted numerically by timestamp**
- Columns can be grouped into **column families**
 - When written, each Column Family has its own directory on the HDFS (.../table/CF_x/...)
 - Allows for grouping of columns frequently accessed together
 - Improved performance when accessing data
 - Within a Column Family, all data for a row stored together on disk

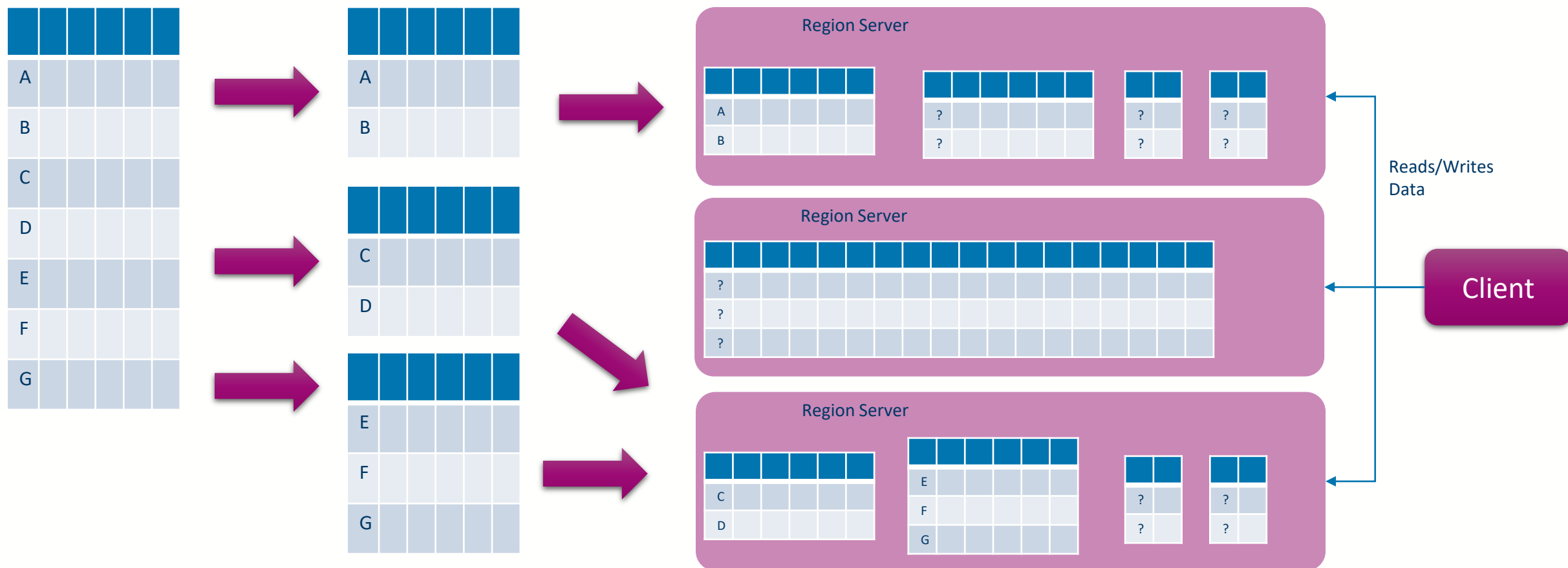


Versions and Time

- Each table cell may have multiple versions
 - Hence the need for a timestamp
 - A timestamp can be actual time or some user-defined quantity (e.g. a version number)
 - Latest version of each cell returned by default
 - Versions of a table cell may be spread out amongst regions
- Users can also specify
 - Explicit number of max versions for the system to keep
 - Explicit max age (e.g. one week)
 - Allows for a sort of “garbage collection”

Data Distribution

- Tables are divided into regions, regions are distributed amongst region servers





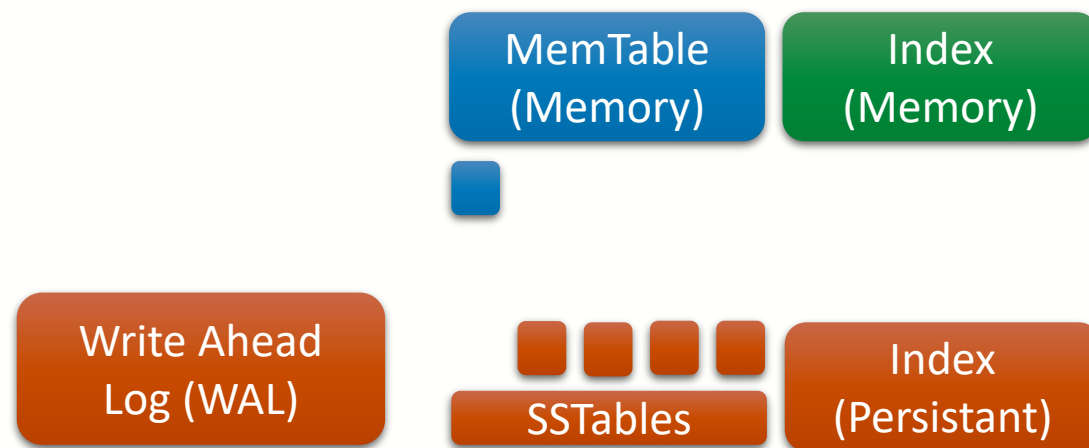
Region Servers

- Responsible for the actual storage of regions (parts of a table)
 - One region server holds multiple regions
- A region server uses 4 storage structures
 - **Write-Ahead Log (WAL)**: Persistently stores write records between spill file writes
 - **MemStore**: Stores the most recent batch of write records in memory. Records are keyed to the files they are about. Read requests check this first.
 - **BlockCache**: Stores a copy of recently read data to improve access time for repeat reads
 - **HFile**: A compressed representation of a set of write records. The MemStore is converted to an HFile when full

This seems familiar...

Log-Structured-Merge (LSM) Tree Components

- **Write Ahead Log (WAL)**: Persistently stores write records between spill file writes
- **MemTable**: Stores the most recent batch of write records in memory. Records are keyed to the files they are about. Read requests check this first.
- **SSTable**: Comprised of a series of spill files. Once the MemTable becomes full, it gets sorted, converted to a spill file and added to the SSTable.
- **Index**: Contains a mapping between file keys and what (persisted) SSTables contain information about that key
- **Compactor**: Merges SSTables to reduce fragmentation of information about a key



Log-Structured-Merge (LSM) Tree Components

- **Write Ahead Log (WAL)**: Persistently stores write records between spill file writes
- **MemTable**: Stores the most recent batch of write records in memory. Records are keyed to the files they are about. Read requests check this first.
- **SSTable**: Comprised of a series of spill files. Once the MemTable becomes full, it gets sorted, converted to a spill file and added to the SSTable.
- **Index**: Contains a mapping between file keys and what (persisted) SSTables contain information about that key
- **Compactor**: Merges SSTables to reduce fragmentation of information about a key

MemStore

HFile

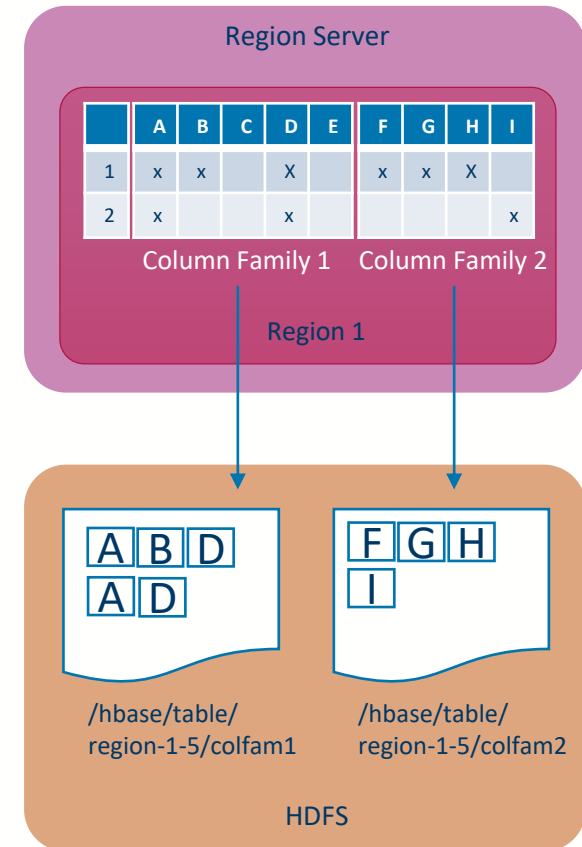
HMaster
does this

Performed on the Region Servers



Region Servers and Column Families

- Each Region is split into Column Families
- A Column Family for a region is the unit of storage within a Region Server
 - Stored in MemStore
 - Persisted to disk as HFiles





Data API

- **Put**(rowkey, column_key, timestamp, value)
- **Get**(rowkey)
 - Returns all cells of row
- **Get**(rowkey, column_key, timestamp)
 - Returns a specific cell
- **Get**(List<Get> gets)
 - Returns a list of results, one per Get operation in the input list
 - A.k.a. a multi-get
- **Scan**(start_rowkey, end_rowkey)
 - Returns all rows with keys in [start_key, end_key)
- **Delete**(rowkey)
- **Delete**(rowkey, timestamp)
- **Delete**(rowkey, column_family [,timestamp])
- **Delete**(rowkey, column_key [,timestamp])



HBase vs HDFS

- HBase is a **column-orientated data store**, while HDFS is a **file store**
- HBase provides support for **random writes**, which HDFS does not
- HBase performs **incremental writes** as needed, rather than performing immediate writes
- HBase **reads smaller blocks of data** (regions) than HDFS blocks
- HBase operations are typically **lower latency** (faster) than the HDFS equivalents



University
of Glasgow

Course Coordinator
Dr. Richard McCreadie

Email: richard.mccreadie@glasgow.ac.uk
Room: SAWB 304

#UofGWorldChangers



@UofGlasgow